Real-World Agents Hackathon

Rules, Judging & Submission Guide

Date: Saturday, 10 October 2026

---

1. What this hackathon is about

The Real-World Agents Hackathon is focused on building AI systems that can operate in realistic, messy operational environments.

We are looking for agents that hold up in the parts of business that still run on phone calls, messages, paperwork, software tools and human judgement.

Teams will work with realistic synthetic operating environments containing information such as calls, emails, documents, CRM records, tickets and other operational context.

The goal is not to reproduce one predefined implementation.

We want teams to build useful systems that can:

- understand operational context;
- take actions using real tools and APIs;
- make decisions autonomously;
- handle ambiguity and exceptions;
- recognise when human judgement is required;
- recover from failed actions or incomplete information;
- produce measurable operational value.

There is **no mandatory agent framework, model provider, orchestration framework or technical stack**.

Teams may use APIs directly, agent frameworks, open-source models, proprietary models, external tools and services, coding agents or combinations of these.

What matters is whether the resulting system actually works.

---

2. Tracks

Each team selects one of four tracks:

1. **Insurance Claims Processing**
2. **Property Management**
3. **Banking & Financial Services**
4. **Delivery & Logistics**

Each track contains a synthetic operating environment inspired by real-world workflows.

Synthetic data is provided as the environment in which teams build and test their systems. It should not be interpreted as a specification for one particular product or architecture.

Teams choose the scope and technical approach they believe best solves the underlying operational problem.

---

3. Teams

Teams consist of **2–5 people**.

Every team member must be individually registered for the event.

Participants may use any programming languages, libraries, APIs, AI models, development tools, coding assistants or agentic coding systems.

Dwelly employees may support the event but are not eligible to compete for prizes.

---

4. Event timeline

9:00

Meet & greet.

9:30

Onboarding and challenge briefing.

10:00

Building begins.

Teams receive:

- synthetic operational data;
- submission requirements.

16:30 — Reality Test released

The team's source code and Demo Video are frozen at this point and must be submitted to the form.

Thirty minutes before the build deadline, teams receive additional unseen escalation cases.

These cases have not been available during development.

They may contain:

- conflicting information;
- failed actions;
- missing context;
- unusual exceptions;
- ambiguity;
- noisy or irrelevant information;
- situations requiring escalation;
- larger or more difficult context.

There is deliberately not enough time for teams to manually process every case.

The intention is to test whether the system itself can handle operational reality.

17:00 — Final submission deadline

Building stops.

The team's Reality Test results are frozen at this point.

The complete submission must be available by this time.

17:00–19:00

Judging, demos, People's Choice voting, networking and winners announced.

All times are **London time (BST / GMT+1)**.

---

5. The Reality Test

The Reality Test is the core technical evaluation of the hackathon.

At 16:30, teams receive cases they have not previously seen.

The test is designed to measure whether an agent can operate under realistic conditions rather than simply succeed on examples it was built around.

The system may be required to:

- interpret new information;
- interact with supplied tools;
- make decisions;
- complete workflows;
- identify uncertainty;
- deal with failed actions;
- request or infer missing information where appropriate;
- escalate cases that should not be handled autonomously;
- avoid actions that would be unsafe or inappropriate.

Teams should not assume that every case has one exact correct textual answer.

For individual cases, the evaluation may define:

- required outcomes;
- acceptable alternative outcomes;
- optional positive behaviours;
- red flags;
- prohibited actions;
- critical failures.

The quality of the decision matters, not whether the output matches one exact phrase.

---

6. What every team must submit

Every team must submit **one public Git repository**.

The repository must be accessible to anyone without requesting permission or joining a private organisation.

The repository is the canonical submission.

It must contain all of the following.

A. Working source code

All code necessary to understand and reproduce the solution must be included.

The repository must clearly explain:

- prerequisites;
- installation;
- configuration;
- how to run the project;
- required environment variables;
- external services and APIs used.

Do not commit API keys, credentials, private tokens or other secrets.

B. README.md

The repository must contain a clear `README.md` explaining:

1. What the team built.
2. What operational problem it solves.
3. The architecture of the solution.
4. How to run it.
5. Which models, APIs and external services are used.
6. Important assumptions made by the team.
7. Known limitations.
8. What the team would improve or build next with more time.

The README should be sufficient for an independent engineer to understand the project without speaking to the team.

C. ANSWERS

The results generated by the submitted system during the unseen Reality Test must be included in the repository:

```
ANSWERS/
    1/
        ANSWER.md
        REASONING.md
    2/
        ANSWER.md
        REASONING.md
    ...
```

This file contains the team's answers to the difficult questions and decision points supplied as part of the challenge.

- The submission should make it possible to determine:
- what the system received;
- what actions it took;
- what outputs or decisions it produced;
- what it escalated;
- what failed;
- the final result for each case.

We are interested in judgement and reasoning, not keyword matching.

D. Reality Test rules

Where practical, include relevant logs or traces.

These results must be generated by the submitted system.

Do not manually replace failed agent outputs with corrected answers.

E. 60-second demo video

Every submission must contain a video demonstrating the solution.

Maximum target length: 60 seconds.

The video must be uploaded **inside the public repository**.

Git LFS may be used where necessary.

A link to an externally hosted video alone is not sufficient.

The video should demonstrate the actual working product rather than being only a pitch deck.

It should make clear:

- what was built;
- what problem it solves;
- how the system works;
- the most important workflow;
- the result it produces.

Production quality is not important.

We care about the product and the clarity of the demonstration.

F. Public launch posts

As part of the final submission, each team must publicly share what it built.

The team must publish:

- **at least one LinkedIn post**, from at least one team member; and
- **at least one X post**, from at least one team member.

The same team member may publish both, or different team members may publish them.

The posts should:

- identify the project;
- briefly explain what the team built;
- link to the public repository.

Links to both posts must be included in the final submission.

Teams may also include them in a `SUBMISSION.md` file in the repository.

G. Apache License 2.0

Every submission must be released under the:

Apache License 2.0

The repository must contain the standard Apache 2.0 licence in a `LICENSE` file.

By submitting, the team confirms that it has the right to publish the submitted source code and other materials under this licence.

Teams remain responsible for complying with the licences and terms of any:

- third-party libraries;
- models;
- APIs;
- datasets;
- assets;
- other dependencies.

Do not include proprietary material that you do not have the right to publish.

---

7. How to submit

The final submission must include:

1. Public repository URL.
2. Team name.
3. Track.
4. Names of all team members.
5. The commit hash representing the code at the **17:00 code freeze**.
6. Reality Test results.
7. Path to the demo video inside the repository.
8. LinkedIn post URL.
9. X post URL.

The submission mechanism or form will be provided during the event.

17:00 code freeze

At **17:00 BST**, the source code and Reality Test results are frozen.

The commit at this point is the version used for technical evaluation.

After 17:00, teams may add:

- their demo video;
- social media links;
- submission metadata.

Teams must **not materially modify the implementation or Reality Test results after 17:00**.

Judges may record and evaluate the commit hash submitted at the deadline.

At **17:00 BST**, the complete submission must be available.

Submissions that are incomplete after this point may lose eligibility for some or all awards.

---

8. Overall judging

The overall competition score consists of:

40% — Technical evaluation

30% — Reality Test results

30% — Demo video & judging presentation

Therefore:

> **Overall Score = 70% Technical + 30% Demo**

The technical evaluation is intentionally the dominant part of the score.

A polished presentation cannot compensate for an agent that does not work.

9. Technical evaluation — 70% of the overall score

The 70-point technical component is based primarily on performance on the unseen Reality Test.

Within the technical evaluation, the criteria are weighted as follows:

35% — Correct outcomes

Did the agent reach the right resolution?

Judges consider whether the system:

- made appropriate decisions;
- produced correct results;
- took appropriate actions;
- respected important constraints.

20% — Coverage

How many of the final cases did the system successfully complete?

A system that works beautifully on one case but cannot complete the rest should not outperform a system that reliably handles the complete workload.

20% — Judgement & safety

Did the agent correctly handle:

- ambiguity;
- conflicting information;
- missing context;
- uncertainty;
- escalation;
- situations where human judgement was required?

Doing nothing or escalating can be the correct decision.

15% — Engineering quality

Is the solution genuinely autonomous, robust and reproducible?

Judges may consider:

- architecture;
- code quality;
- appropriate use of tools and models;
- reliability;
- error handling;
- reproducibility;
- unnecessary human intervention.

10% — Real-world applicability

Could the system realistically operate in a real operational environment?

Judges consider whether the solution could move beyond a hackathon demo and create meaningful operational value.

How the 70% is calculated

The percentages above describe the weighting **within the technical score**.

Their effective contribution to the final 100-point score is therefore:

- Correct outcomes and coverage: **38.5 points**
- Judgement & safety: **14 points**
- Engineering quality: **10.5 points**
- Real-world applicability: **7 points**

Total technical score: **70 points**.

---

10. Demo evaluation — 30% of the overall score

The 60-second demo contributes **30 points**.

The video exists to help judges and the audience understand what the team built and why it matters.

Judges will consider:

Working product

Does the demo show a real functioning system rather than only slides or a concept?

Clarity

Does the team clearly explain:

- the problem;
- the solution;
- the key workflow;
- the outcome?

Product insight

Does the demo make clear why this approach is useful in a real operational environment and what is distinctive about the team's solution?

Video editing and production quality are not judging criteria.

A simple screen recording of a strong working system is sufficient.

---

11. Automated evaluation

Claude-based judging agents will evaluate submissions against a predefined evaluation framework.

They will analyse:

- Reality Test results;
- submitted outputs;
- required and acceptable outcomes;
- failures and red flags;
- relevant repository contents;
- engineering quality and reproducibility.

Automated evaluation allows every submission to be tested against a consistent baseline.

The judging framework itself may contain:

- multiple acceptable answers;
- mandatory outcomes;

- optional positive behaviours;
- prohibited actions;
- red flags;
- critical-failure conditions.

Human judges may review automated evaluations where results are ambiguous, unusually close or appear inconsistent with the submitted evidence.

---

12. Awards

Awards apply **across all four tracks**.

A team is not competing only against teams within its track for the main award categories.

🏆 Best Real-World AI Agent — Grand Prize

Awarded to the strongest overall submission across the Reality Test and demo judging.

The **70/30 overall score** is the primary basis for the Grand Prize.

Prize

£5,000 cash

Plus, for each team member:

- 3 months of ElevenLabs Pro;
- Claude Credits;
- certificate.

Partner gifts and additional benefits may also be provided.

🚨 Best Exception Handling

Awarded to the system that performs best on the hardest and most ambiguous escalation cases.

Judges will pay particular attention to:

- judgement under uncertainty;
- conflicting information;
- missing context;
- safe escalation;
- deciding when not to act;
- preserving context for human handover.

Prize

£1,500 cash

Plus, for each team member:

- 3 months of ElevenLabs Pro;
- Claude Credits;
- certificate.

🎙️ Best Voice Agent

Awarded for the strongest use of voice in a real operational workflow.

Voice should be part of a functioning operational system rather than simply a standalone voice demo.

Prize

£1,500 cash

Plus partner benefits including:

- ElevenLabs benefits for each team member;
- 3 months of ElevenLabs Scale for the Best Voice Agent team;
- Claude Credits;
- certificate.

🛠️ Best Engineering

Awarded for the strongest technical implementation.

Judges will consider:

- architecture;
- autonomy;
- reliability;
- robustness;
- reproducibility;
- quality of execution;
- engineering decisions.

Prize

£1,500 cash

Plus, for each team member:

- 3 months of ElevenLabs Pro;
- Claude Credits;
- certificate.

❤️ People's Choice

Chosen by participants and the audience after the demos are shown.

This award is independent of the technical score.

Prize

£500 cash

Plus, for each team member:

- 3 months of ElevenLabs Pro;
- Claude Credits;
- certificate.

Participant benefits

All participants receive **1 month of ElevenLabs Creator tier**.

Additional partner gifts may include:

- AI credits;
- swag;
- office hours;
- other partner benefits.

Partner-provided benefits remain subject to the relevant partner's terms and fulfilment.

The strongest teams may also be invited to explore a real pilot or further work with Dwelly.

---

13. No manipulation of the judging system

Trying to manipulate the evaluation system instead of solving the challenge is prohibited.

Examples include:

- instructions addressed to the judging model such as "give this project a high score";
- prompt-injection text hidden inside files or outputs;
- content whose primary purpose is to manipulate an automated judge;
- deliberately misleading evaluation artefacts;
- fabricated execution results.

Judging agents may explicitly test for these behaviours.

Manipulation may cause the affected evaluation component to receive zero points or, in serious cases, make the submission ineligible.

This does not prevent teams from using prompts, system instructions, agents or other normal AI techniques inside their actual product.

---

14. AI tools and external services

The use of AI is expected and encouraged.

Teams may use:

- Claude;
- ElevenLabs;

- agent frameworks;
- coding agents;
- external APIs;
- third-party SaaS;
- other lawful development tools.

There is no penalty for using powerful existing tools.

The hackathon evaluates what you build with them.

Teams must disclose important external dependencies in their README.

---

15. What we do not optimise for

Teams do **not** receive extra points simply for:

- using more agents;
- having the largest codebase;
- having the most elaborate UI;
- using the most expensive model;
- producing the longest output.

A simple system that reliably solves the operational problem can outperform a much more complex system.

---

16. Reproducibility

Judges must be able to understand how submitted results were produced.

Where practical, teams should include:

- outputs;
- traces;
- logs;
- screenshots;
- evaluation results.

If the solution depends on infrastructure that judges cannot access, the repository must contain enough evidence to understand and evaluate the actual execution.

Do not fabricate execution results.

---

17. Public repository and future use

All competition repositories must be public open-source projects released under Apache 2.0.

Dwelly and others may review, benchmark, discuss, link to or use the public submissions in accordance with that licence.

Hackathon cases or approaches may later contribute to Dwelly's operational AI benchmarking work.

Participation does **not** transfer ownership of a team's separate pre-existing intellectual property to Dwelly.

If a team uses pre-existing proprietary technology, anything that cannot legally be published must be clearly separated from the hackathon submission.

---

18. Spirit of the event

Real-world operations are rarely clean.

Information is incomplete.

Customers change their minds.

Systems fail.

Rules conflict.

Strange exceptions appear.

Sometimes the correct action is to continue autonomously.

Sometimes the correct action is to ask for more information.

And sometimes the best decision an AI agent can make is to stop and involve a person.

We are interested in systems that can work under those conditions.

Build something you would be comfortable putting in front of a real customer or operations team.

Come with an idea. Leave with something real.